

CLI & CRON — Operação Psico-RAG (v0.1)

Documentação prática dos **programas de linha de comando** e das **rotinas em cron** do projeto Psico-RAG. Inclui: o que cada comando faz, parâmetros, pré-requisitos, logs, quando rodar e exemplos de agendamento.

Sumário

- [Visão geral](#)
 - [Pré-requisitos e variáveis de ambiente](#)
 - [Comandos principais \(CLI\)](#)
 - [1\) Ingestão de PDFs — `src.rag.ingest\_pdfs`](#)
 - [2\) Carga de notas privadas — `src.rag.load\_notes`](#)
 - [3\) Treinamento de modelo — `src.models.train`](#)
 - [4\) \(Opcional\) Avaliação/score — `src.models.score`](#)
 - [5\) API HTTP — `uvicorn src.api.main:app`](#)
 - [Rotinas de manutenção](#)
 - [Snapshots/backup do Qdrant](#)
 - [Limpeza de logs e artefatos](#)
 - [Agendamentos \(cron\)](#)
 - [Crontab recomendado \(America/Sao Paulo\)](#)
 - [Boas práticas no cron](#)
 - [Padrões de logging](#)
 - [Solução de problemas](#)
 - [Glossário rápido](#)
-

Visão geral

O pipeline possui duas fontes principais de contexto: 1. **Literatura pública (PDFs)**, vetorizada e armazenada no **Qdrant**. 2. **Notas privadas (prontuário)**, carregadas sob autorização.

Modelos supervisionados podem ser treinados para tarefas específicas (ex.: ranking por turma/risco). A API expõe rotas para **search** e **answer**.

Pré-requisitos e variáveis de ambiente

Crie um arquivo `.env` na raiz do projeto (exemplo):

```
# ===== Qdrant =====  
QDRANT_URL=http://localhost:6333  
QDRANT_API_KEY=  
QDRANT_COLLECTION=literatura  
QDRANT_COLLECTION_NOTES=prontuario
```

```
# ===== Embeddings =====
EMBEDDING_MODEL=text-embedding-3-small
OPENAI_API_KEY=sk-...

# ===== Caminhos =====
PDF_DIR=./data/pdfs
NOTES_CSV=./data/notes/notes.csv # ou Parquet

# ===== Treinamento =====
MODEL_DIR=./models
TRAIN_CFG=./config/config.yaml
```

Ajuste conforme sua infraestrutura (Docker, paths, chaves). Em produção, prefira **variáveis de ambiente do sistema** ou um cofre (AWS Secrets Manager), evitando expor chaves.

Comandos principais (CLI)

1) Ingestão de PDFs — `src.rag.ingest_pdfs`

O que faz: Lê PDFs da pasta `PDF_DIR`, gera **chunks** e **embeddings**, e indexa/atualiza a coleção pública no Qdrant.

Execução

```
python -m src.rag.ingest_pdfs
```

Parâmetros (via env) - `PDF_DIR`: diretório com PDFs. - `QDRANT_URL`, `QDRANT_COLLECTION`, `EMBEDDING_MODEL`, `OPENAI_API_KEY`.

Saída/efeitos - Cria/atualiza a coleção `QDRANT_COLLECTION` com pontos (vetores + payload de metadados). - Loga progresso (nº de documentos, chunks, latência por lote de embeddings).

Quando rodar - Após adicionar/atualizar PDFs. - Agendado **diariamente** de madrugada se há atualização frequente de acervo.

Erros comuns - *OOM/timeout de embeddings*: reduza `batch_size` ou limite de páginas. - *Conflito de coleção*: confirme nomes/métricas no Qdrant.

2) Carga de notas privadas — `src.rag.load_notes`

O que faz: Lê CSV/Parquet de notas (prontuário), faz pré-processamento (limpeza, anonimização necessária) e indexa na coleção privada `QDRANT_COLLECTION_NOTES`.

Execução

```
python -m src.rag.load_notes
```

Parâmetros (via env) - `NOTES_CSV` (ou Parquet), `QDRANT_URL`, `QDRANT_COLLECTION_NOTES`, `EMBEDDING_MODEL`, `OPENAI_API_KEY`.

Saída/efeitos - Indexa trechos/linhas do prontuário, com payload mínimo (ex.: `class_id`, datas, tags de atendimento).

Quando rodar - Após novas anotações ou batch de atualizações do CPPE. - Agendado **em janelas fora do expediente** (LGPD: princípio da minimização — indexar somente o necessário).

Erros comuns - *Arquivo grande em máquina pequena*: considerar **maior memória** ou processar em chunks.

3) Treinamento de modelo — `src.models.train`

O que faz: Treina/atualiza o **modelo supervisionado** (ex.: XGBoost, Sklearn) com dados de features preparados pelo ETL do projeto.

Execução

```
python -m src.models.train --cfg ./config/config.yaml
```

Parâmetros - `--cfg`: caminho do YAML com hiperparâmetros, splits, paths de dados e saída em `MODEL_DIR`.

Saída/efeitos - Salva artefatos (ex.: `model.joblib`, métricas em `.json` / `.csv`).

Quando rodar - Após mudanças relevantes no dataset/eng features. - **Semanal** ou **mensal**, conforme cadência de dados.

Erros comuns - `ModuleNotFoundError`: ative o venv correto e instale `requirements.txt`. - *Conflitos de versões* (ex.: `numpy<2`): alinhe versões pinadas.

4) (Opcional) Avaliação/score — `src.models.score`

O que faz: Carrega o modelo treinado e produz métricas/relatórios de validação (ex.: AUC, F1, SHAP). (Se este módulo existir no repositório — mantenha nome/assintura conforme seu projeto.)

Execução (exemplo)

```
python -m src.models.score --cfg ./config/config.yaml --out ./reports
```

Quando rodar - Após o treinamento, antes de promover o modelo a produção.

5) API HTTP — `uvicorn src.api.main:app`

O que faz: Sobe a API FastAPI com rotas `/health`, `/search`, `/answer`.

Execução (dev)

```
uvicorn src.api.main:app --host 0.0.0.0 --port 8000 --reload
```

Execução (prod, systemd) Crie um serviço `psico-rag-api.service` apontando para o virtualenv, com `Restart=always`.

Quando rodar - Ambientes de teste e produção.

Rotinas de manutenção

Snapshots/backup do Qdrant

Opção A — API de snapshots do Qdrant

Crie snapshot da coleção e copie para storage externo/S3.

Opção B — volume Docker

Se o Qdrant roda em Docker com `-v ~/qdrant_storage:/qdrant/storage`, compacte diretórios de snapshot exportados.

Importante: Use **janela de baixa escrita** e/ou **modo somente leitura** durante snapshot para consistência.

Limpeza de logs e artefatos

- Rotacionar logs em `/var/log/psico-rag/*.log` (logrotate).
 - Remover snapshots/artigos antigos conforme política de retenção.
-

Agendamentos (cron)

Crontab recomendado (America/Sao_Paulo)

Use `crontab -e` para o usuário de execução (ex.: `psicorag`). Utilize `flock` para evitar concorrência.

```
# ==== Variáveis globais do cron
SHELL=/bin/bash
PATH=/usr/local/bin:/usr/bin:/bin
```

```

# Diretório do projeto
PROJECT_DIR=/opt/psico-rag-poc
VENV=$PROJECT_DIR/.venv/bin/activate
LOG_DIR=/var/log/psico-rag

# 1) Ingestão de PDFs (diário, 02:15)
15 2 * * * . $VENV && cd $PROJECT_DIR &&
    flock -n /tmp/ingest_pdfs.lock
    bash -lc "python -m src.rag.ingest_pdfs >> $LOG_DIR/ingest_pdfs.log 2>&1"

# 2) Carga de notas privadas (dias úteis, 02:45)
45 2 * * 1-5 . $VENV && cd $PROJECT_DIR &&
    flock -n /tmp/load_notes.lock
    bash -lc "python -m src.rag.load_notes >> $LOG_DIR/load_notes.log 2>&1"

# 3) Treino semanal (domingo 03:30)
30 3 * * 0 . $VENV && cd $PROJECT_DIR &&
    flock -n /tmp/train.lock
    bash -lc "python -m src.models.train --cfg ./config/config.yaml >>
$LOG_DIR/train.log 2>&1"

# 4) Snapshot do Qdrant (diário, 03:50)
50 3 * * * . $VENV && cd $PROJECT_DIR &&
    flock -n /tmp/qdrant_snapshot.lock
    bash -lc "./scripts/qdrant_snapshot.sh >> $LOG_DIR/qdrant_snapshot.log
2>&1"

# 5) Limpeza de logs (mensal, dia 1 às 04:10)
10 4 1 * * find $LOG_DIR -type f -name '*.log' -mtime +60 -delete

```

Ajuste horários para sua janela de manutenção. Se usar **systemd timers**, replique a lógica acima em unidades `.service`/`.timer` com `ConditionACPower`, `After=network-online.target`, etc.

Boas práticas no cron

- **flock** evita instâncias simultâneas.
- **Logs separados** por tarefa facilitam depuração.
- **Alertas**: integre saída do cron com `mailx` ou webhook (ex.: `ntfy`) em caso de erro (`||` envia alerta).
- **LGPD**: agendar carga de notas em janelas com menor risco e monitorar acessos.

Padrões de logging

- Cada tarefa escreve em `LOG_DIR`.
- Prefixar logs com timestamp: configure seu logger (ex.: `loguru`) ou `ts` do `moreutils`.
- Em produção, considere **journald + loki/promtail + Grafana**.

Solução de problemas

- **Conflito de container Qdrant:** `docker ps -a`, `docker rm -f qdrant` ou use `--name qdrant2`. Configure `--restart unless-stopped` para subir no boot.
 - **Memória insuficiente (2 GB):** processe em **lotes menores**, aumente swap, ou migre para instância com 4–8 GB.
 - **Falha de rede Qdrant:** verifique `-p 6333:6333` e políticas de firewall/SG.
 - **Erros de versão Python/pacotes:** pinagem em `requirements.txt`; evite duplicatas.
-

Glossário rápido

- **Ingestão:** processo de converter PDFs em *chunks* + *embeddings* e indexar no vetor DB.
 - **Snapshot:** export de estado da coleção para restauração/backup.
 - **flock:** utilitário para controle de exclusão mútua em scripts de cron.
 - **LGPD — minimização:** indexar/usar apenas o necessário para a finalidade.
-

Nota: Se algum módulo tiver nome/assinatura diferente no seu repositório (ex.: `score`), padronize o título acima e ajuste os comandos. Caso deseje, adicione scripts shell em `./scripts/` com todas as chamadas e *health checks* prontos para uso.